

SecShare

Vulnerable App — Mimari Tasarım Dökümanı

Spring Boot 3.4 · JWT · PostgreSQL | Feature-flag'li · Bağlı saldırı zinciri · API + frontend
Hazırlayan: Claude Code | Tarih: 2026-07-09 | Durum: Tasarım (kod yazılmadı)

Keşif — Mevcut Durum

Uygulama: Spring Boot 3.4 REST API — JWT auth + PostgreSQL + dosya paylaşımı. Katmanlar: `auth` (login/register, JWT filter, SecurityConfig), `file` (upload/download/delete, ownership check), `user`, `config`.

İlginç olan: Uygulama şu an aslında **oldukça güvenli** yazılmış — bu bir avantaj. Sağlam bir baseline üzerine kontrollü açık ekleyeceğiz:

- Path traversal koruması var (`normalize()` + `startsWith` kontrolü)
- Dosya erişiminde ownership kontrolü var (`getOwnedFileOrThrow`)
- BCrypt(12), extension allowlist, boyut limiti
- JWT HS256 imza doğrulaması yapılıyor

Zaten mevcut olan zayıflıklar (recon'da hemen göze çarpanlar):

#	Zayıflık	Yer
A	Hardcoded JWT secret git'e commit'lenmiş	<code>application.properties:3</code>
B	JWT filter rolleri DB'den değil token claim'inden okuyor → secret ele geçerse admin forge	<code>JwtAuthenticationFilter</code>
C	<code>/h2-console/**</code> , <code>/test.html</code> , <code>/files.html</code> <code>permitAll</code>	<code>SecurityConfig</code>
D	Register 409 "Email already exists" → user enumeration	<code>AuthService</code>
E	Login'de rate limit yok → brute force	<code>AuthService</code>
F	Login her zaman <code>List.of("USER")</code> hardcoded ediyor (ADMIN hiç üretilmiyor)	<code>AuthService:64</code>

⚠ **Not:** `AuthService.java` 'da commit'lenmemiş bir değişiklik build'i kırıyor — satır 44'te `Instant.now()` şeklinde bölünmüş. Uygulama aşamasında ilk iş bunu düzeltmek.

1. Feature-Flag Mekanizması

`application-vuln.properties` (profil: `vuln`) — her açık tek satırla aç/kapa:

```
# Master switch
vuln.enabled=true
# Bireysel açıklar
vuln.jwt.weak-secret=true           # bilinen/zayıf HMAC secret
vuln.jwt.allow-none=false          # alg:none kabulü (opsiyonel/zor)
vuln.auth.user-enum=true           # register/login farklı cevap
vuln.auth.no-rate-limit=true       # brute force serbest
vuln.access.legacy-download=true   # IDOR endpoint aktif
vuln.search.sqli=true              # native concat query
vuln.file.path-traversal=true       # filename ile indirme
vuln.import.ssrp=true              # URL fetch
vuln.import.xxe=true               # XML parser DTD açık
vuln.process.cmd-injection=true     # Runtime.exec
vuln.serve.stored-xss=true          # inline content-type
vuln.cors.wildcard=true            # Access-Control-Allow-Origin *
vuln.errors.verbose=true           # stacktrace sızıntısı
```

`@ConfigurationProperties("vuln")` ile `VulnProperties` bean'i; her zafiyetli bileşen `@ConditionalOnProperty` ile kayıtlanır. **Flag kapalıyken güvenli davranış** (before/after eğitimi için kritik).

2. Seed Data

`db/seed` + `CommandLineRunner` :

Kullanıcı	Şifre	Rol	İçindeki "flag" dosyası
<code>admin@secshare.local</code>	güçlü (bulunmamalı)	ADMIN	<code>master-keys.txt</code> (final flag)
<code>alice@corp.local</code>	<code>Summer2024!</code> (zayıf, brute)	USER	<code>salary.pdf</code>
<code>bob@corp.local</code>	<code>password123</code>	USER	<code>db-backup.txt</code> (DB creds → SSRF pivot)

Not: `admin` 'e login ile ulaşılamaz (güçlü şifre) — **sadece JWT forge ile** erişilir. Bu, zincirin kilit noktası.

3. Bağlı Saldırı Zinciri (Ana Senaryo)

Her aşama bir sonrakinin anahtarını üretir:

Aşama 0 — Recon. Yüzey haritası: `/actuator/env` (expose edilecek), `/h2-console`, `/test.html`,

/swagger-ui . → actuator/env veya git history'de `app.jwt.secret` görünür.

Aşama 1 — User Enumeration + Brute Force. `vuln.auth.user-enum` : register/login farklı cevaplar.
`vuln.auth.no-rate-limit` : sınırsız deneme → `alice` / `bob` şifreleri kırılır. → Normal USER token elde edilir.

Aşama 2 — JWT Zayıf Secret → PrivEsc. `vuln.jwt.weak-secret` : leak edilmiş zayıf secret ile `roles:["ADMIN"]` token forge. Filter rolü token'dan okuduğu için ADMIN olunur. → `GET /api/files/all` .

Aşama 3 — IDOR. `vuln.access.legacy-download` : `GET /api/files/legacy/{id}` ownership check'siz. → `bob` 'un `db-backup.txt` = DB creds.

Aşama 4 — SQL Injection. `vuln.search.sqli` : `GET /api/files/search?name=` native concat query.
→ `users` tablosundan email + bcrypt hash dump → offline crack.

Aşama 5 — Path Traversal / LFI. `vuln.file.path-traversal` : `GET /api/files/raw?path=` normalize'siz. → `../../../../application.properties` , `/etc/passwd` , kaynak kod.

Aşama 6 — SSRF. `vuln.import.ssrf` : `POST /api/files/import {url}` sunucu URL'i çeker. → iç ağ
/ `169.254.169.254` metadata / `h2-console` pivot.

Aşama 7 — Stored XSS. `vuln.serve.stored-xss` : `GET /api/files/view/{id}` inline + kullanıcı content-type. → `.svg` / `.html` upload → JS çalışır (admin oturumu çalınır).

Aşama 8 — Command Injection / RCE (final). `vuln.process.cmd-injection` : `POST /api/files/archive {filename}` → `Runtime.exec("zip out.zip "+filename) . → ; cat master-keys.txt` = final flag / RCE.

Bonus açıklar: `XXE (XML import)` `Zip Slip` `Mass assignment (register roles)` `CORS *` `Verbose stacktrace`

4. Frontend Dokunuşları (Minimal)

- `files.html` : arama kutusu (SQLi tetikler), "URL'den içe aktar" formu (SSRF), dosya listesini `innerHTML` ile basma (DOM XSS + stored XSS render noktası).
- `test.html` : JWT decode/görüntüleme yardımcısı (recon kolaylığı) — bırakılabilir.

5. Modül / Dosya Yerleşimi

```
vuln/
├─ VulnProperties.java      @ConfigurationProperties
├─ VulnConfig.java         bean kayıt + CORS filter
```

```
|— search/FileSearchController.java + SearchRepository (native query)
|— legacy/LegacyFileController.java IDOR + path traversal (raw)
|— importer/ImportController.java    SSRF + XXE
|— process/ArchiveController.java    command injection
|— serve/ViewController.java         stored XSS
|— seed/DataSeeder.java              CommandLineRunner
```

Mevcut `auth / file` koduna dokunuş: `JwtService` (flag'li zayıf secret), `AuthService` (enum/rate-limit), `SecurityConfig` (yeni endpoint'ler + actuator expose). Hepsi flag arkasında.

6. Dokümanlar

- `docs/CHALLENGES.md` — kursiyer için: her aşama başlığı, hedef, "ne bulmalısın", kademeli ipuçları (spoiler'sız), OWASP referansı.
- `docs/SOLUTIONS.md` — eğitmen için: her aşamanın tam exploit adımları (curl/PoC), örnek forge token, beklenen çıktı, flag değerleri.
- `README-VULN.md` — kurulum: `SPRING_PROFILES_ACTIVE=vuln`, H2 modu.

7. Uygulama Sırası

1. Build fix (`AuthService` typo) + `vuln` profili + `VulnProperties` iskeleti
2. Seed data + H2 dev opsiyonu (çalışır hedef)
3. Auth zafiyetleri (enum, rate-limit, weak-secret) → Aşama 1–2
4. IDOR + SQLi + Path traversal → Aşama 3–5
5. SSRF/XXE + XSS + Command injection → Aşama 6–8
6. Frontend + dokümanlar
7. SECURE mode doğrulaması (tüm flag'ler false → sızma testleri fail olmalı)

Karar Noktaları

- **DB:** Docker'sız kolay demo için `vuln` profilinde H2 (in-memory + seed) önerilir; PostgreSQL prod baseline kalır.
- **XXE:** SSRF ile aynı import modülünde tutuluyor; istenmezse çıkarılabilir.